

Position and Altitude Control of a Quadrotor with Single Motor Failure

Team 24

Abstract

Mid-flight motor failure in a quadrotor UAV results in uncontrolled and chaotic flight of the drone, which ultimately causes it to crash into potentially populated regions, risking damage to the UAV and posing safety hazards. Despite the severity of this issue, limited directly implementable open-source solutions exist to counter it. This report investigates a fault-tolerant control framework for quadrotor UAVs that ensures stable hover, controlled descent, and navigation despite a single motor failure. The problem is addressed by detecting motor failure and switching from the default controller to a custom controller. This report presents a complete software-based solution requiring no additional sensors and is easily deployable on commercially available quadrotor airframes with the PX4 autopilot. New modules are developed in the firmware to handle detection and control while minimal changes are made to the existing modules. A Kalman Filter-based probabilistic Loss-of-Effectiveness (LOE) estimator is implemented for fault detection, along with a Primary Axis Non-Linear Dynamic Inversion (PA+NDI) controller within the PX4 autopilot firmware. Software-in-the-loop (SITL) simulations are performed in the Gazebo environment, followed by tests on a custom drone to validate the performance on hardware.

Contents

1	Introduction	1
2	Nomenclature	2
3	Problem Understanding	2
4	Literature Survey	3
4.1	Motor failure detection:	3
4.2	Fault-tolerant control:	3
5	Experimental Setup	4
5.1	Simulation	4
5.2	Hardware	4
6	Motor Failure Detection	5
7	Fault-Tolerant Controller Design	7
7.1	Inner loop control	7
7.2	Outer loop control	8
8	Software Architecture	8
8.1	PX4 Software Stack	8
8.2	Motor Failure Injection	9
8.3	Motor Failure Detection	10
8.4	Control Law Implementation	10

9	Results	10
9.1	Motor Failure Detection	10
9.2	Control Law Performance	12
10	Incorporation of Midterm Feedback and Performance Enhancements	13
11	Challenges and Lessons Learned	14
12	Conclusion	14
13	Improvements & Applications	14
	References	15

1. Introduction

Unmanned Aerial Vehicles (UAVs), especially quadrotors, have gained widespread use across various sectors due to their ability to perform critical tasks such as aerial surveillance, mapping, reconnaissance, and emergency response. Their compact design, vertical takeoff and landing (VTOL) capabilities, along with stable hovering, make them versatile platforms for both civilian and defense applications. A quadrotor UAV is an under-actuated system that is fully controllable. However, even a single motor failure causes a loss of controllability, potentially resulting in crashes. Therefore,

fault-tolerant flight control is a viable solution as it requires only software adaptation, effectively avoiding the need for rotor redundancy, addition of parachutes, or other hardware fail-safe mechanisms.

In the midterm report, we introduced our solution pipeline for motor failure detection and recovery of a quadrotor after motor failure. We discussed the detection algorithms and control laws along with the results achieved. Building upon the previous work, we have updated our failure detection algorithm and control law based on robustness to changes in system characteristics. We then verified their performance through MATLAB and SITL simulations using the PX4 Autopilot firmware.

2. Nomenclature

The following nomenclature has been employed throughout the report:

Table 1. Nomenclature

m	Mass of the Quadrotor
$I_{x,y,z}$	Moment of Inertia about x,y,z axis
K_f	Motor Force coefficient
K_m	Motor Moment coefficient
$p_{x,y,z}$	x,y,z component of drone's position
$v_{x,y,z}$	Velocity of drone in x,y,z direction
$a_{x,y,z}$	Linear acceleration of drone in x,y,z direction
ϕ, θ, ψ	Roll, pitch, yaw of the drone respectively
p, q, r	x, y, z component of drone's angular velocity
h_i	Distance of motor i from COM along x
b_i	Distance of motor i from COM along y
ω_i	Rotation speed of rotor i
F_i	Thrust generated by the rotor i
T	Combined thrust force in the body frame
X^B	Vector X as expressed in body frame
X^W	Vector X as expressed in world frame
G	Control Effectiveness Matrix of the drone
u_i	Control input to rotor i

3. Problem Understanding

A quadrotor has 6 degrees of freedom (DOF): roll, pitch, yaw, and translational degrees of freedom in X, Y, and Z directions. However, it only has 4 actuators. So, the degrees of freedom in X and Y are achieved by coupling the remaining rotational and translational degrees of freedom. This enables us to completely control the drone using only 4 motors. The relation between the thrust force, torques, and control inputs to the drone in the body frame can be given by the relation mentioned in Equation 1

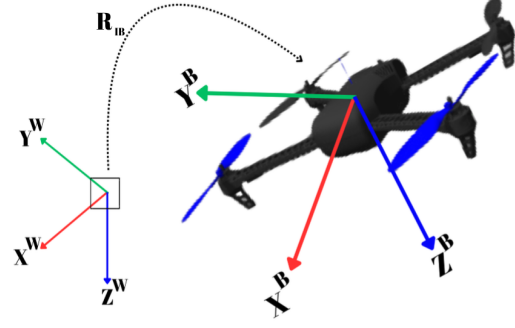


Figure 1. World frame and Body frame by convention

$$\begin{bmatrix} \tau_x \\ \tau_y \\ \tau_z \\ T \end{bmatrix} = G \begin{bmatrix} u_1^2 \\ u_2^2 \\ u_3^2 \\ u_4^2 \end{bmatrix} \quad (1)$$

where G is the control effectiveness matrix defined for fig:quad as,

$$G = \begin{bmatrix} -K_f.b_1 & K_f.b_2 & K_f.b_3 & -K_f.b_4 \\ K_f.h_1 & -K_f.h_2 & K_f.h_3 & -K_f.h_4 \\ K_m & K_m & -K_m & -K_m \\ -K_f & -K_f & -K_f & -K_f \end{bmatrix} \quad (2)$$

Here, the control input u_i is defined as the normalized rotor speed of the i^{th} rotor, so that $u_i = \omega_i / \omega_{max}$ and $u_i \in [0, 1]$, since all the motors are assumed to be unidirectional; ω_{max} is the maximum rotor speed. Since ω_{max} is constant for a given setup, $\omega_i = Cu_i$.

When one of the rotors fails, a quadrotor loses the ability to independently control one of the variables out of roll, pitch, yaw, and altitude. Roll and pitch are used coupled with altitude to control X and Y, making them essential for having a controlled flight. Thus, the only feasible option is to give up control over the yaw (called the heading of the vehicle).

A general quadrotor typically includes basic sensors such as an accelerometer, gyroscope, magnetometer, barometer, and GPS. To ensure that rotor failure detection is widely applicable, we must rely solely on these basic sensors. Only highly specialized models have ESC (Electronic Speed Controller) feedback, which can directly indicate rotor failure. These ESCs may not be commercially viable, and the solution should be economically viable and, thus, not use the ESCs. However, when a rotor fails, the readings of the basic sensors become extremely noisy, making it challenging to analyze and

predict the system's behaviour.

To maintain stability in such a scenario, the quadrotor must detect the failure and stop sending commands to the failed motor. This rapid detection is essential to keep the system within the small-angle approximation— an assumption fundamental to standard quadrotor dynamics. Large deviations make the system difficult to control and may result in a complete loss of stability (due to the possibility of flipping over). Therefore, it is crucial to implement a detection algorithm that enables the quadrotor to respond to failure quickly, preserving control and minimizing destabilizing effects.

Losing a rotor leads to a loss in torque control (due to the non-reversibility nature of the rotors) in the x-y plane, as one of the pairs can only provide torque in a particular direction. Thus, a fault-tolerant controller should be designed to allocate outputs to the remaining three rotors.

PX4, with its real-time capabilities and hardware compatibility, is well-suited for implementing the control and failure detection algorithms. However, since it only has basic built-in motor failure handling, both motor failure detection and a fault-tolerant controller must be developed. While direct modification of the firmware could utilize its full functionalities, it may interfere with other firmware components, potentially reducing the overall capabilities of the PX4 Autopilot. Therefore, the solution pipeline should be designed to ensure no interference between different components, enable switching between custom and standard controllers in the event of motor failure, maintain safety features, and employ a modular implementation for ease of deployment. Additionally, the coding style should maintain open-source standards to allow for further contributions and developments.

4. Literature Survey

4.1 Motor failure detection:

The general approach to detect failure is to monitor the motor's health as a state and implement a Kalman filter to estimate the same. The failure is ascertained by using thresholding on this state. Multiple implementations of the Kalman filter with variations in state and/or bias estimating equations are available. Chapter 44-47 of [1] summarizes Kalman Filter-based approaches for failure detection approaches including Kalman Filter, Extended Kalman Filter (EKF), Unscented Kalman Filter (UKF) and Multi-Model Adaptive Estimation (MMAE). Two-Stage Kalman filter (TSKF) [2] is used to simultaneously estimate and isolate possible faults in each

motor. The algorithm runs 2 Kalman filters that estimate the control effectiveness vector and state, respectively and uses coupling equations to revise the state estimate. The control effectiveness vector ranges from 0 (healthy motor) to 1 (complete failure). The above approach was further refined for robustness in [3] by Adaptive TSKF. It clips the covariance values within a limit to avoid unprecedented random noise. The effectiveness vector is passed through a statistical test to report failure. UKF implemented in [4] provides robust state estimation by using the unscented transform, where a set of sigma points captures the mean and covariance of the system, which are then propagated through the non-linear dynamics. Unlike the EKF, it avoids linearization, improving accuracy in highly non-linear systems, but requires higher computation and poses potential sensitivity to the process and measurement noise covariances. [5] presents a simplified approach of detecting failure without state estimation but utilizing the feedback angular accelerations and linear acceleration in z-direction in the body frame as measurement vector. A low-pass filter has been introduced to handle noisy measurements. The actuator effectiveness parameters k_i on Kalman filtering and a hypothesis test are used to measure the probability of failure. The detailed description of the algorithm is described in the following sections.

4.2 Fault-tolerant control:

There are 2 major approaches to designing a control on a system: classical and adaptive. Classical approaches include PID or H_{∞} loop shaping techniques [6], whereas adaptive methods include techniques like ML and RL-based approaches [7]. A linear feedback approach to design an FTC was described in [8] on a non-linear state space system. The authors have shown that to have full control of roll, pitch and altitude, we need to sacrifice yaw. However, if yaw control is sacrificed, then we cannot expect a bounded yaw rate. In practicality, inaccuracies are developed in gyroscope measurements, which can prove disastrous. [9] have used an PA+NDI (Primary Axis-Non-Linear Dynamic Inversion) control approach towards the problem. They have shown that the yaw rate can be bounded if the drone is allowed to wobble (gyroscopic effect). [6] considers the application of Linear Parameter Varying (LPV) modelling and control techniques with the yaw rate acting as a time-varying scheduling parameter. Finally, [7] discusses how recursive neural networks can be used to tune PID controllers and compares the results with well-known tuning techniques like Ziegler-Nichols and extremum seeking.

5. Experimental Setup

5.1 Simulation

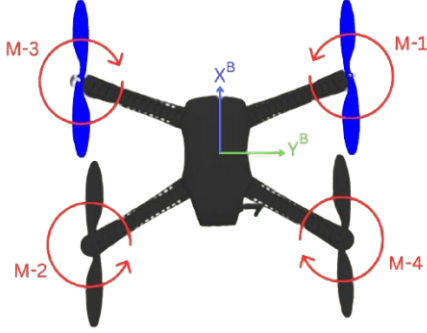


Figure 2. Iris drone used in simulation

Before deploying our algorithm on hardware, it is essential to validate our approach on simulation. To ensure our simulation closely resembles real-world conditions, we selected the Gazebo Classic physics simulator, as recommended. As for the drone, we tested our approach on a generic model of the iris quadrotor in the ‘X’ configuration. This iris model contains only the most rudimentary sensors- gyroscope, accelerometer, IMU, GPS sensor and magnetometer. Table 2 enlists the various parameters of the iris quadrotor. The mass of the quadcopter was reduced to 0.85 kg from 1.5 kg, as enough thrust was not available for the same by only 2 rotors.

Table 2. Table of parameters of quadrotor for simulation

Parameter	Unit	Value
I_x	kg.m/s ²	0.029
I_y	kg.m/s ²	0.029
I_z	kg.m/s ²	0.055
m	kg	0.850
K_f	N	7.920
K_m	N.m	0.475
h_1, h_2, h_3, h_4	m	0.130
b_1, b_3	m	0.22
b_2, b_4	m	0.20

For high-level navigation of our quadrotor including take-off, landing and waypoint traversal, the QGroundControl (QGC) has been employed. It provides an interactive interface for mission planning and drone setup. Both Gazebo and QGC use the MAVLINK communication protocol to interface with the PX4 Autopilot. Figure 3 illustrates the SITL communication pipeline.

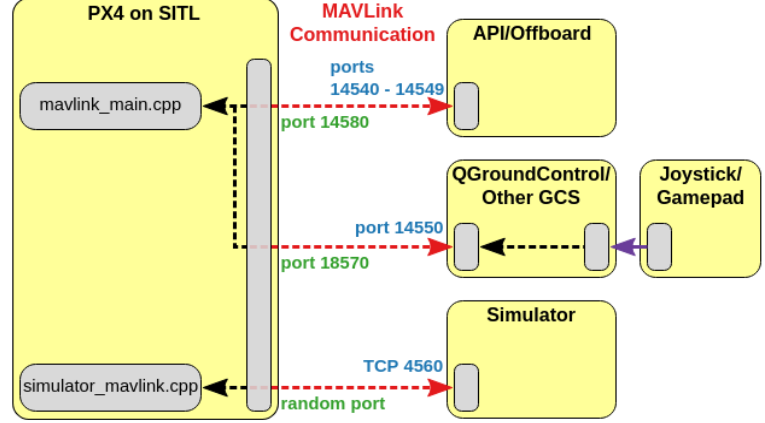


Figure 3. Gazebo-PX4 interface, referenced from [10]

5.2 Hardware

A symmetric ‘X’ configured frame was selected. Component selection prioritized crash resilience, a favorable thrust-to-weight ratio, and reliable control stability. The various components used are:

- **Motor:** Emax ECOII-2306-1700KV Brushless Motor
- **GPS:** Holybro M10 Micro
- **Flight Controller (FC):** Pixhawk Cube Orange
- **Receiver:** RP1 V2 ExpressLRS Nano Receiver
- **ESC Stack:** SpeedyBee BLS 60A 30x30 4-in-1 ESC
- **Propellers:** Gemfan Flash 7040 Tri-Blade 7” Props
- **Frame:** GEPRC Mark 4 v2 8-inch
- **Battery:** 1500mAh 6S 120C LiPo Battery
- **Telemetry:** Holybro Sik Telemetry Radio V3-100 mW 433 MHz



Figure 4. Drone used for hardware testing

Table 3 lists the relevant physical parameters of the quadrotor. The values of mass and inertia were obtained by replicating the experiment as described in [11]. For measuring inertia,

the drone was suspended from two points using strings in different configurations, as required. For I_z , the strings were tied to the base frame, and after arming the drone with motors off, small angle oscillations were performed such that the drone behaved like a torsional pendulum. The following formula was used to obtain the inertia:

$$I_z = \frac{mgD^2}{4d(2\pi f)^2} = \frac{mgD^2T^2}{16\pi^2d} \quad (3)$$

The time period for rotations was obtained after analyzing the log files. Similarly, for I_x and I_y , the small angle oscillations were performed such that the drone could be approximated as a simple pendulum to estimate inertia about the point of suspension. The parallel axis theorem was then used to obtain the final values of I_x and I_y . For the time period, the log files were referred to, as done previously. The following formula was used to obtain the inertia:

$$T = 2\pi\sqrt{\frac{I_s}{mgd}} \quad (4)$$

$$I_s = I_b + md^2 \quad (5)$$

where,

D : Distance between two suspension points

d : Distance between suspension points and COM of drone

T : Time period of one oscillation

m : Mass of the drone

g : Acceleration due to gravity

I_b : I_x or I_y of the drone about center of mass

I_s : I_x or I_y of the drone about point of suspension

K_f was calculated using the following relation between the control inputs and the weight of the drone:

$$K_T = \frac{\text{weight(kg)}}{u_1^2 + u_2^2 + u_3^2 + u_4^2} \quad (6)$$

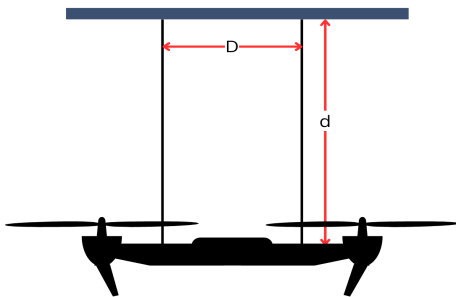


Figure 5. Setup

Table 3. Table of parameters of quadrotor for hardware

Parameter	Unit	Value
I_x	kg.m/s ²	0.034
I_y	kg.m/s ²	0.015
I_z	kg.m/s ²	0.050
m	kg	0.843
K_t	N	6.567
K_m	N.m	0.821
b_1, b_2, b_3, b_4	m	0.110
h_1, h_2, h_3, h_4	m	0.150

6. Motor Failure Detection

Fault Detection and Isolation (FDI) is recognized as a critical component of autonomous systems. In quadrotors, faults arising from rotor failure are often attributed to motor malfunction (e.g., overheating or wear), ESC failure, propeller damage, or unpredictable environmental factors. The analysis and isolation of such failures can be facilitated by ESC feedback. However, in the absence of ESC feedback, the evolution of the state in response to the failure needs to be analyzed. For motor failure detection, a subset of the state vector is selected, comprising parameters that exhibit significant variance under failure conditions. In the midterm submission, the methodology described in [2] was implemented. However, upon critically analyzing its performance across a range of edge cases, limitations were identified. Consequently, a refined approach, based on [5], was proposed. This section outlines the new approach in detail, followed by a comparative analysis of the two methodologies and a justification for the change.

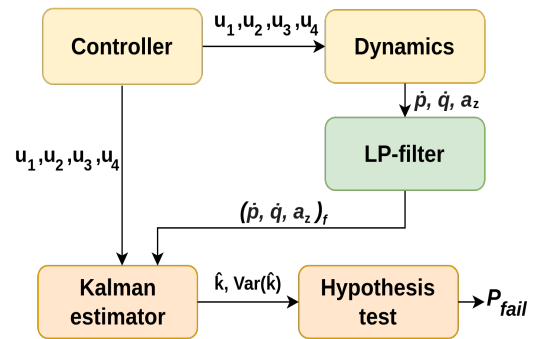


Figure 6. Dataflow for Failure Detection

To achieve reliable motor failure estimates, the Kalman Filter-based Loss of Effectiveness (LoE) estimation algorithm proposed in [5] is implemented. Motor failure is determined by estimating the control effectiveness vector $\mathbf{k} = [k_1, k_2, k_3, k_4]$, where $F_i = K_f k_i u_i^2$ represents the force exerted by each rotor.

The drone dynamics can be modelled as follows:

$$\begin{bmatrix} \dot{p} \\ \dot{q} \\ \dot{a}_z \end{bmatrix} = K_f \begin{bmatrix} \frac{-b_1}{I_x} & \frac{b_2}{I_y} & \frac{b_3}{I_x} & \frac{-b_4}{I_y} \\ \frac{h_1}{I_y} & \frac{-h_2}{I_y} & \frac{h_3}{I_y} & \frac{-h_4}{I_y} \\ \frac{-1}{m} & \frac{-1}{m} & \frac{-1}{m} & \frac{-1}{m} \end{bmatrix} \text{diag}(\mathbf{k}) \begin{bmatrix} u_1^2 \\ u_2^2 \\ u_3^2 \\ u_4^2 \end{bmatrix} \quad (7)$$

With $\mathbf{z} = [\dot{p}, \dot{q}, \dot{a}_z]$ as the measurement vector and $\mathbf{x}$ as the predicted control effectiveness $[\hat{k}_1, \hat{k}_2, \hat{k}_3, \hat{k}_4]$, the model can be re-written as:

$$\mathbf{z} = H(\mathbf{u})\mathbf{x} \quad (8)$$

where,

$$H = \begin{bmatrix} -G_{p1}u_1^2 & G_{p2}u_2^2 & G_{p3}u_3^2 & -G_{p4}u_4^2 \\ G_{q1}u_1^2 & -G_{q2}u_2^2 & G_{q3}u_3^2 & -G_{q4}u_4^2 \\ -G_{az}u_1^2 & -G_{az}u_2^2 & -G_{az}u_3^2 & -G_{az}u_4^2 \end{bmatrix} \quad (9)$$

$$G_{pi} = \frac{K_f b_i}{I_x}, G_{qi} = \frac{K_f h_i}{I_y}, G_{az} = \frac{K_f}{m} \quad (10)$$

Our approach differs from the one proposed in [5] in that we have used control inputs to formulate our model while [5] uses ω^2 . Although the latter is a more accurate model since the relation between ω and u can only be approximated as linear, determining the actual ω on board is only possible using sophisticated ESCs. Thus, for practical reasons, we chose to instead employ the control inputs in our dynamics, assuming the mixer dynamics to be linear.

Our algorithm performed consistently on simulation. However, the sensor data obtained on board the hardware drone proved too noisy to be utilized unprocessed in the Kalman Filter update equations. Therefore, the linear and angular accelerations are processed using a first-order low-pass filter, defined by the update equation:

$$out_k = out_{k-1} + \alpha \cdot (in_k - out_{k-1}) \quad (11)$$

where α the update parameter is determined experimentally, out_k the filter output at the time step k , and in_k the unfiltered data.

The following describes the iterative algorithm for estimating the control effectiveness vector:

Algorithm 1 Kalman Filter Update

Input: $\mathbf{x}_{k-1}, P_{k-1}, \mathbf{z}_k$
 $P_{k|k-1} = P_{k-1} + Q$
 $\mathbf{y} = \mathbf{z}_k - H_k \mathbf{x}_{k-1}$
 $S = R + H_k P_{k|k-1} H_k^\top$
 $K = P_{k|k-1} H_k^\top S^{-1}$
 $\mathbf{x}_k = \mathbf{x}_{k-1} + K \mathbf{y}$
 $P_{k|k} = (I - K H_k) P_{k|k-1}$
return $\mathbf{x}_k, P_{k|k}$

where,

P = State Covariance Matrix
 Q = Process Noise Covariance Matrix
 R = Measurement Noise Covariance Matrix
 $\mathbf{y}$ = Measurement Error Vector
 $\mathbf{z}$ = Measurement Vector
 $\mathbf{x}$ = State Vector
 K = Kalman Gain
 S = Residual Covariance

$\mathbf{x}_k$ contains the estimated actuator effectiveness, $\hat{k}_i$, for each of the rotors. A loss in actuator effectiveness below a threshold level, k_{thresh} is suggestive of a motor failure. The probability of failure for each rotor, $P_{fail,i}$ under the Gaussian distribution assumption of $\mathbf{x}_k$ is defined as:

$$P(\hat{k}_i < k_{thresh}) = \frac{1}{2} + \frac{1}{2} \left[\text{erf} \left(\frac{k_{thresh} - \hat{k}_i}{\sigma_{k_i} \sqrt{2}} \right) \right] \quad (12)$$

Here, $\sigma_{k_i}^2$ are the diagonal terms of the state covariance matrix, P . Since the injected failure is a complete rotor failure, $P(\hat{k}_i < k_{thresh}) = 1$ during failure. In reality, since $P_{fail,i}$ never reaches unity, we consider $P_{fail,i} > P_{thresh} = 1 - \epsilon$ as the indication of motor failure, where ϵ is a small experimentally determined constant.

In order to further reduce the chances of false-positive detections, we define the parameter *min_counts* as the minimum number of iterations for which $P_{fail,i}$ should remain above the threshold level for the algorithm to assert a motor failure.

The values of Q and R are determined following a sensitivity analysis-based approach, wherein the parameters are varied one at a time, and the set of values corresponding to the minimum detection time are tabulated. In order to speed up this process of parameter tuning, this process is first run offline on a package such as MATLAB, using flight data logged on our custom drone; the optimal parameters determined using sensitivity analysis are then used for online estimation, as described in Figure 7.

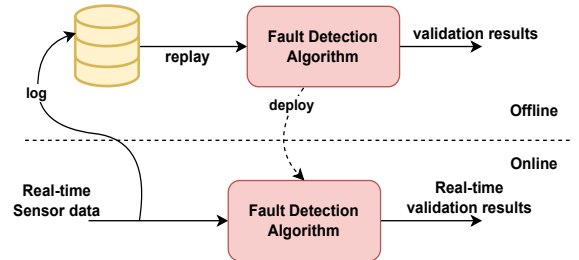


Figure 7. Offline validation based parameter tuning

This approach has been preferred over our previously attempted approach based on [2] majorly due to its resilience to the pre-failure state of the drone. It has proven to be reliable under majority of flight conditions, including significant pre-failure pitch, roll and yaw speeds. Moreover, this estimator is robust to a large range of model uncertainties with minimal tuning. The same wasn't true for our former approach; although its performance was accurate under hover conditions, the results worsened with non-zero roll, pitch angles, and yaw rate. This result is at par with our expectations; since [2] utilizes a linearized KF, it fails to handle non-linearities by design. Our final implementation, on the other hand, does not need to linearize dynamics since our state variables are accelerations that relate to control input by a linear transformation. Furthermore, [2] utilizes the complete 12-dimensional state vector in its EKF, which makes it much more computationally expensive than [5], which has just 4 state variables.

7. Fault-Tolerant Controller Design

7.1 Inner loop control

The current controller implementation draws major inspiration from [9]. During mid-term, we had two parallel implementations in the pipeline [8, 9]. With better results from PA+NDI compared to feedback linearization, we moved forward and implemented the PA+NDI controller in PX4.

The position controller computes the desired acceleration, denoted by $\mathbf{a}_{\text{des}}^{\text{W}}$, in the world frame. The direction of the required thrust, represented by $\mathbf{n}_{\text{des}}$, is determined by following relation:

$$\mathbf{n}_{\text{des}}^{\text{W}} = \frac{\mathbf{a}_{\text{des}}^{\text{W}} - \mathbf{g}}{\|\mathbf{a}_{\text{des}}^{\text{W}} - \mathbf{g}\|} \quad (13)$$

where $\mathbf{g} = [0, 0, g]$ is oriented along the positive $\mathbf{z}^{\text{W}}$ axis.

Note that $\mathbf{n}_{\text{des}}^{\text{W}}$ is in world frame. To ensure consistency in the control law, we transform $\mathbf{n}_{\text{des}}^{\text{W}}$ from the world frame to the drone frame when calculating the error dynamics.

A fixed vector $\mathbf{n}_{\text{fix}}^{\text{B}}$ in the drone frame, which is tilted from the $\mathbf{z}^{\text{B}}$ -axis by a predefined angle θ , serves as the desired thrust axis in the drone frame. This tilt is introduced to prevent excessive yaw rate. The θ is tunable. If the i^{th} motor, with coordinates $\{x_i, y_i\}$, fails, the fixed vector in the body frame

can be described as:

$$\mathbf{n}_{\text{fix}}^{\text{B}} = (n_x^{\text{B}}, n_y^{\text{B}}, n_z^{\text{B}}) \quad (14)$$

$$n_x^{\text{B}} = \sin \theta \left(\frac{-x_i}{\sqrt{x_i^2 + y_i^2}} \right) \quad (15)$$

$$n_y^{\text{B}} = \sin \theta \left(\frac{-y_i}{\sqrt{x_i^2 + y_i^2}} \right) \quad (16)$$

$$n_z^{\text{B}} = \sqrt{1 - (n_x^{\text{B}})^2 - (n_y^{\text{B}})^2} \quad (17)$$

The current orientation of the drone frame is required to convert $\mathbf{n}_{\text{des}}$ from the world frame to the drone frame. This is achieved by measuring the quaternions corresponding to the quadrotor's orientation and applying the following transformation:

$$\mathbf{n}_{\text{des}}^{\text{B}} = \text{DCM} \cdot \mathbf{n}_{\text{des}}^{\text{W}} \quad (18)$$

where DCM is the Direction Cosine Matrix, which is the rotation matrix that transforms vectors from the world frame to the drone frame. where $[q_0, q_1, q_2, q_3]$ is the attitude quaternion.

The error between the desired thrust orientation and the fixed orientation vector in the drone frame is defined as:

$$\mathbf{y} = \mathbf{n}_{\text{des}}^{\text{B}} - \mathbf{n}_{\text{fix}}^{\text{B}} \quad (19)$$

For stability of the second-order closed-loop dynamics of $\mathbf{y}$, the control input $\tilde{u}$ must follow:

$$\ddot{\mathbf{y}}(\tilde{u}) = -\text{diag}(k_{p_1}, k_{p_2}) \mathbf{y} - \text{diag}(k_{d_1}, k_{d_2}) \dot{\mathbf{y}}, \quad (20)$$

where k_{p_i} and k_{d_i} are the proportional and derivative gains, respectively. With $\mathbf{n}_{\text{des}}^{\text{B}} = [n_x, n_y, n_z]$, the following assumptions are made to simplify the calculations:

- The yaw rate, r , and the z component of $\mathbf{n}_{\text{des}}$, n_z , are assumed to be constant after motor failure.
- The desired orientation of thrust vector $\mathbf{n}_{\text{des}}^{\text{W}}$ changes slowly, such that $\dot{\mathbf{n}}_{\text{des}}$ can be assumed to be $\mathbf{0}$.

With these assumptions, the commanded torques can be computed as:

$$\begin{bmatrix} \tau_{x,\text{cmd}} \\ \tau_{y,\text{cmd}} \end{bmatrix} = \begin{bmatrix} [-k_{p_1}y_2 + k_{d_1}(n_x r - n_z p) + (n_y r - n_z q)r]I_x/n_z \\ + I_z q r - I_y q r \\ [k_{p_2}y_1 + k_{d_2}(-n_z q + n_y r) + (n_x r - n_z p)r]I_y/n_z \\ + I_x p r - I_z p r \end{bmatrix} \quad (21)$$

The $\mathbf{z}^{\text{B}}$ axis is tilted at some angle α with the $\mathbf{z}^{\text{W}}$ axis, such that $\cos(\alpha) = \cos(\theta)\cos(\phi)$. The desired thrust is then

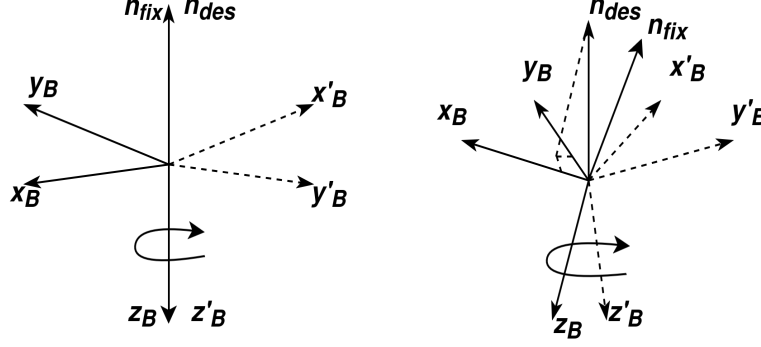


Figure 8. a) $\mathbf{n}_{des}^B$ is aligned to $\mathbf{n}_{fix}^B$ hence the drone will not wobble and only have yaw.
b) $\mathbf{n}_{des}^B$ is not aligned to $\mathbf{n}_{fix}^B$, so the drone will wobble, but at a lower yaw rate

given by:

$$T_{cmd} = -\frac{m(a_{z,des} - g)}{\cos \theta \cos \phi}, \quad (22)$$

where m is the mass of the drone and g is the acceleration due to gravity.

Since yaw torque is not directly controllable, the corresponding row (third row) in the effectiveness matrix is removed. Additionally, the column corresponding to the failed motor is also removed, resulting in a reduced 3x3 matrix. The inverse of this reduced matrix is computed, and the output from the controller must be a 4x1 vector. Given that the i^{th} motor has failed, the reduced matrix is augmented by adding zeros in the i^{th} row and the third column, corresponding to the missing motor and yaw torque, respectively. The control inputs are then computed by multiplying the inverse of this zero-appended reduced effectiveness matrix, $\tilde{G}$, with the commanded torques and thrust:

$$[u_1^2, u_2^2, u_3^2, u_4^2]^T = \tilde{G}^{-1} [\tau_{x,cmd}, \tau_{y,cmd}, 0, T_{cmd}]^T \quad (23)$$

This formulation ensures that the system can tolerate motor failures while maintaining stable and effective control.

7.2 Outer loop control

Our outer loop controller is essentially a PD controller that sets the acceleration set points for the inner loop. The acceleration setpoints are obtained from the following equation :

$$a_{x,des} = k'_{p1}(p_{x,sp} - p_x) + k'_{d1}(v_{x,sp} - v_x) \quad (24)$$

$$a_{y,des} = k'_{p2}(p_{y,sp} - p_y) + k'_{d2}(v_{y,sp} - v_y) \quad (25)$$

$$a_{z,des} = k'_{p3}(p_{z,sp} - p_z) + k'_{d3}(v_{z,sp} - v_z) \quad (26)$$

We have also implemented the functionality of controlling the quadrotor using an RC after motor failure in the simulator,

this can be achieved by simply transforming the $\mathbf{z}_{des}$ axis to:

$$\mathbf{z}_{des} = [-\sin(\theta)\cos(\phi), -\sin(\phi), \cos(\theta)\cos(\phi)], \quad (27)$$

where $\mathbf{z}_{des}$ represents the vector pointing toward the desired roll and pitch angles.

8. Software Architecture

8.1 PX4 Software Stack

PX4 Autopilot is one of the most widely used flight control software due to its modular architecture, compatibility with a wide range of flight controllers, and an intuitive internal communication bus (uORB)[12]. Our implementation strives to best integrate our custom flight detection and controller pipeline with PX4's existing pipeline while making minimal changes to the core PX4 modules. This ensures that the performance of the drone under nominal conditions remains unaffected by our modifications. Figure 9 illustrates and summarizes our entire pipeline.

The following is a high-level overview of some important PX4 modules, along with any additional changes:

8.1.1 Core Controller Modules

The core PX4 controller can be broadly subdivided into inner and outer loops: the position and velocity controllers implemented in the `mc_pos_control` module constitute the outer loop, which operates at 125Hz by default, while the attitude and angular rate controller implemented in `mc_att_control` and `mc_rate_control` respectively constitute the inner loop operating at 250Hz.

- **mc_pos_control:** This module is responsible for the outer loop execution. It uses a P-PID controller with feedforward terms [13] for computing acceleration setpoints. It takes as input the desired trajectory via the

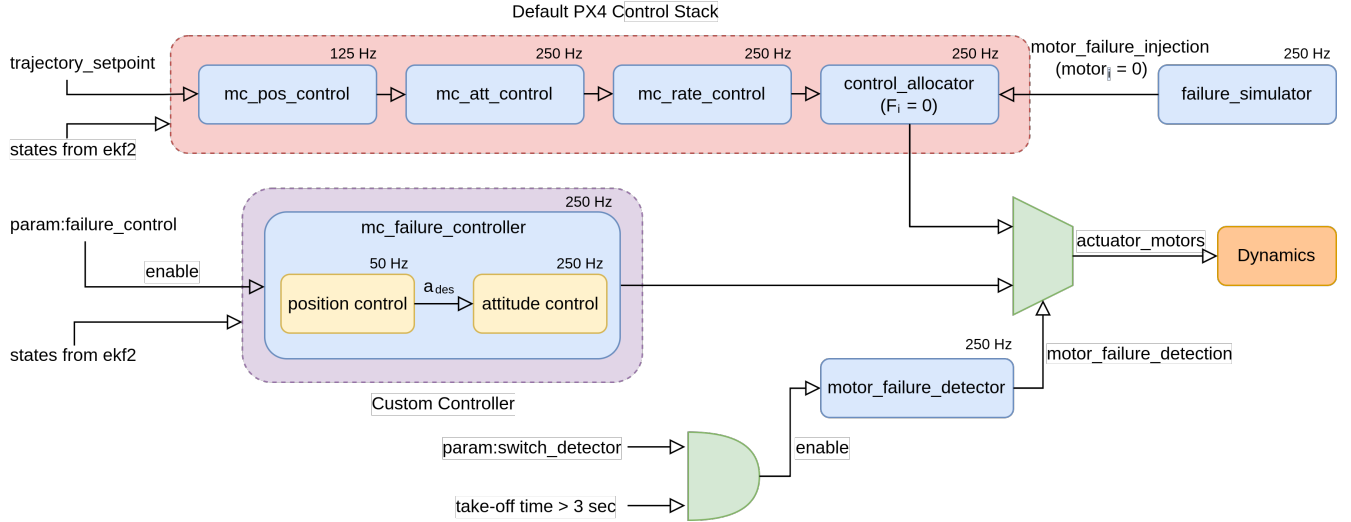


Figure 9. Software Pipeline

trajectory_setpoint uORB topic and publishes the attitude quaternion setpoint and commanded thrust to the mc_att_control module via vehicle_attitude_setpoint topic. Further, it publishes setpoint position, acceleration, and velocity on the local_position_setpoint topic.

- **mc_att_control:** A non-linear quaternion-based controller [14] is implemented in this module to compute the rate setpoints. These setpoints are published to the mc_rate_control module.
- **mc_rate_control:** A simple PID control on rate setpoints generates torque setpoints. Along with the torque, thrust setpoints are published to the control_allocator module via vehicle_torque_setpoint and vehicle_thrust_setpoint topics.
- **control_allocator:** It subscribes to the thrust and torque setpoints, pre-multiplies it to an airframe-dependent control-effectiveness matrix and, publishes the normalized rotor angular speed setpoints to the mixer on the actuator_motors topic. Implementing failure injection in our implementation requires changes to this module described in subsection 8.2.

8.1.2 State Estimation Module

- **ekf2:** PX4 implements an Extended Kalman Filter (EKF) based state estimator in this module. It subscribes to the raw sensor data from the onboard sensors and publishes the drone's estimated states on the vehicle_local_position, vehicle_angular_velocity & vehicle_attitude topics, among others.

8.1.3 Custom Modules

- **mc_failure_controller:** The PA+NDI based custom controller described in subsection 8.4 is implemented in this module. It has the functionality of executing high level position control (e.g. using QGC) as well as lower level roll-pitch control (e.g. using joystick). It subscribes to local_position_setpoint for trajectory input and to the manual_control_setpoint topic for the roll, pitch and thrust setpoints from the joystick or Radio Transmitter. It then publishes actuator setpoints via the actuator_motors topic.
- **motor_failure_detection:** The Kalman Filter based probabilistic estimator described in section 6 is implemented here. It subscribes to vehicle_local_position, vehicle_angular_velocity, and a custom actuator_setpoint topic for the current state data. After executing the detection algorithm, data indicating motor failure is published on the custom motor_failure_detection topic.
- **failure_simulator:** This module is responsible for injecting a motor failure. It publishes the motor failure data on the motor_failure_injection topic.

All of our custom modules operate at 250 Hz.

8.2 Motor Failure Injection

In order to inject failure in a particular motor, we start the failure_simulator module with a parameter that indicates the motor we desire to fail for our simulation. The control_allocator module subscribes to this topic and sets the corresponding actuator setpoint to null before publishing. Although the initially proposed injection mechanism involved the use of the

gazebo_motor_failure plugin, which demanded the use of a ROS2 topic to inject failure, we chose to proceed via the former method since this is more generalized and easy to implement on the hardware using the MAVLINK console, making it independent of extra requirements. Despite using the same mechanism to inject failure on our hardware setup as well, our detection algorithm is robust enough to handle failure detection in hardware as well, a claim that is well validated in the section 9.

8.3 Motor Failure Detection

Our choice of the failure detection algorithm is justified in section 6, and its performance is validated in subsection 9.1. The algorithm proposed in section 6 is implemented as it is. Rather than using the rotor speeds, we rely on the control inputs from the control_allocator module. As described earlier, our method of failure injection required us to set one of the control inputs to null before publishing. Thus, a custom topic has been employed to publish the actuator setpoints before this operation, for use in our algorithm. The output of this algorithm is responsible for dynamically switching to our custom controller. An elementary switching mechanism has been employed for the same:

- The detection data is subscribed by the control_allocator module to conditionally stop publishing the actuator commands from the default controller
- The mc_failure_controller module subscribes to this topic to begin its execution and to start publishing actuator commands from our custom controller.

At the time of the first takeoff, the data from the various sensors spike up instantaneously. This spike can potentially be interpreted as a motor failure since our algorithm has a direct dependence on drone states. In order to deal with this corner case, we begin our detection module at a fixed time after the initial takeoff. Further, the detection module can be disabled by resetting the custom parameter switch_detector in case the user wishes to do so.

8.4 Control Law Implementation

Due to the change in the dynamical model of the drone on motor failure, a custom controller implementation is necessary to solve this problem statement. Further, this module must be receptive to external trajectory setpoints over MAVLINK messages, such as waypoint navigation using QGC and high-level control via custom ROS scripts. Appending this module carefully to the existing stack is also vital to make optimal use

of the current modules. Keeping this in mind, we design the mc_failure_controller module as follows:

- The module subscribes to the local_position_setpoint topic for the desired trajectory. This topic is subscribed compared to other waypoint navigation topics like trajectory_setpoint and trajectory_setpoint_triplet for the following reasons:
 1. mc_pos_control processes the trajectory_setpoint data by application of a PID control on the setpoint error, and generating the local_position_setpoint data by adding the trajectory_setpoint data as a feedforward term. This makes this data more reliable than raw trajectory setpoints.
 2. Existing routines in PX4, such as takeoff, land and various flight modes, can be used without making any additional changes to our controller module since mc_pos_control appropriately adjusts this topic whenever these routines execute. Further, any inconsistencies in the data are accounted for by mc_pos_control, and this topic is updated accordingly.
- A PD control is applied to the position and velocity error to compute the acceleration setpoints while the. This forms the outer loop of our controller that operates at 50Hz. It simply executes once in every five iterations of the module to achieve this reduced frequency.
- The acceleration setpoints determined by the outer loop of mc_failure_controller is input to the PA+NDI algorithm described in section 7. The commanded rotor speeds computed by this algorithm are published on the actuator_motors topic after normalization.
- The execution of the control algorithm within this module is controlled by a parameter failure_control and the motor_failure_detection topic. Thus, whenever the parameter is set and a failure is detected, the module starts publishing actuator setpoints. Conversely, the user can reset the parameter in case the module needs to be disabled.

9. Results

9.1 Motor Failure Detection

The actuator failure detection algorithm described in section 6 is implemented on our PX4 stack as proposed in subsection 8.3. The algorithm is validated on simulation and on our

hardware setup across a variety of flight conditions and uncertainties that occur during a real flight. The various test cases presented in our analysis are: 1)Initial hover state, 2)Non-zero initial roll angle, 3)Non-zero initial pitch angle, and 4)Non-zero initial yaw velocity. The failure controller module had been disabled at the time of testing our detection algorithm.

The plots below incorporate the variation of failure probabilities of the rotors after failure injection and the corresponding detection times. Under nominal hover condition, the P_{fail} for each of the rotors is around 0.5. On injecting failure, the P_{fail} corresponding to the failed rotor begins to rise and finally increases beyond P_{thresh} . Experimentally, a suitable value of P_{thresh} is determined to be 0.9999. Appendix II contains relevant plots for tests performed on simulation and hardware.

Figure 10 illustrates the result of failure injection in motor 1 while the drone is initially in the state of stable hover. As visible in the graph, $P_{fail,1}$ rises and saturates to unity in 160 ms. The other curves initially start rising but achieve a peak value significantly lower than the curve of the failed motor. After the first detection time, the P_{fail} curve of the failed motor remains saturated for a further 0.3 seconds, while the other curves decrease to a much lower value, clearly distinguishing the failed motor from the rest. The evolution in states of the drone in this time period has been plotted in Figure 11 and Figure 12. As observed in the graphs, the roll and pitch angles change by less than ± 10 degrees, which can easily be stabilized by our controller as justified in subsection 9.2. Moreover, there is a minimal variation in the altitude in this time period.

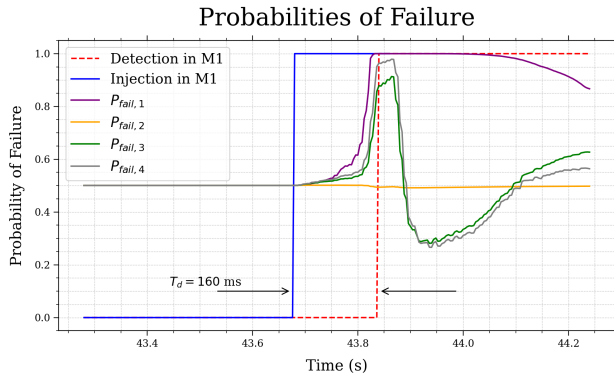


Figure 10. Evolution of probabilities after failure in motor 1

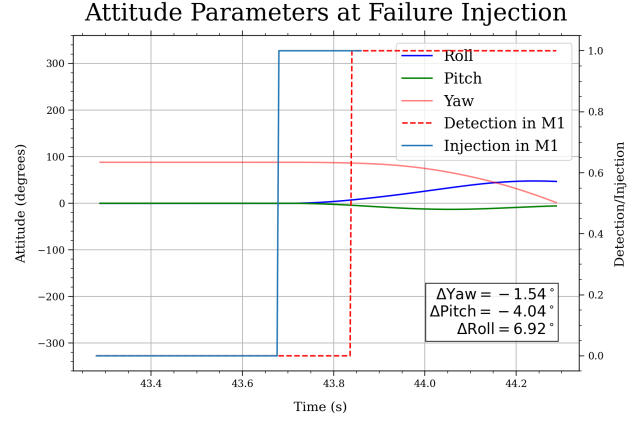


Figure 11. Change in attitude until failure detection

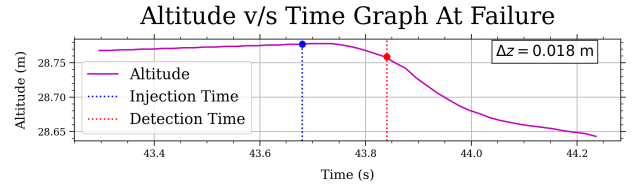


Figure 12. Drop in altitude until failure detection

Figure 23 and Figure 26 show the variation in $P_{fail,i}$ under non-zero initial pitch and roll angles respectively. This condition arises when a motor fails en route a mission. The correctness of the detection algorithms has been verified under these conditions as depicted by the plots. The time for detection increases by a few milliseconds, but still remains lesser than 250 milliseconds. Our algorithm has been successfully verified for initial roll and pitch upto 0.46 rad, which is the maximum controllable attitudes on QGC.

While executing a mission, if a motor fails while the UAV is turning, the drone would have a non-zero yaw rate just before failing. The algorithm is robust enough to cater to this case as well. Figure 29 illustrates this case, wherein the correct motor is detected despite having an initial yaw rate of 1 rad/s. We have verified the accuracy of our algorithm for initial yaw rates upto 2.5 rad/s which is the maximum controllable yaw rate; for most practical applications the drone's yaw rate while turning is lower than this value, thus it can be concluded that our algorithm is robust to initial roll, pitch and yaw rates.

9.2 Control Law Performance

The control law described in section 7 has been implemented and validated through simulation. The performance of the controller is presented in three stages: 1) hover condition, 2) landing, and 3) navigation.

9.2.1 Comparison for hover condition

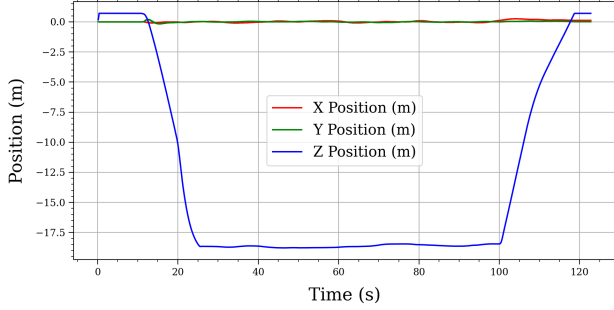


Figure 13. Position in normal hover condition and landing

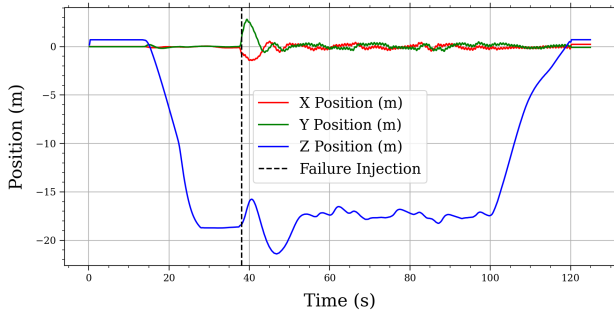


Figure 14. Position after motor failure in hover condition and landing

The hover performance of the controller can be validated by comparing the x^B , y^B and z^B values for the case when the motor failure is not implemented (shown in Figure 13) and when the motor fails and custom controller kicks in (in Figure 14). One thing to note here is that since the data is plotted in the world frame (in NED convention) and the drone goes up in altitude, z becomes more negative.

After failure, the drone's altitude experiences a temporary dip; however, the controller effectively stabilizes the altitude, returning the drone to the desired hover position. The steady-state error is 5% (≈ 1 m), and the recovery time; defined as the time taken for the controller to bring the drone back to 95% of the desired altitude; is 12.2 s.

In Figure 15, we observe that the yaw is 90° under normal hover conditions before motor failure injection. After the motor failure is injected, as seen in Figure 15, the yaw is no longer controlled, and the drone begins to wobble. This results

in roll and pitch oscillations, although they remain bounded within the range of $(-30^\circ, 30^\circ)$.

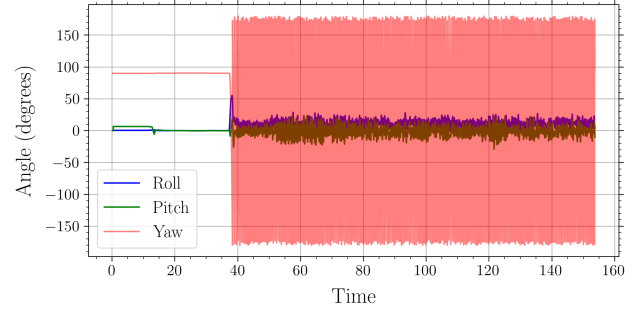


Figure 15. Orientation after motor failure

Additionally, since the drone is wobbling, the pitch and roll rates exhibit oscillations, but these are also bounded within the range $(-7, 1)$ rad/s for pitch rate and $(-4, 4)$ rad/s for roll rate, as can be seen in Figure 16.

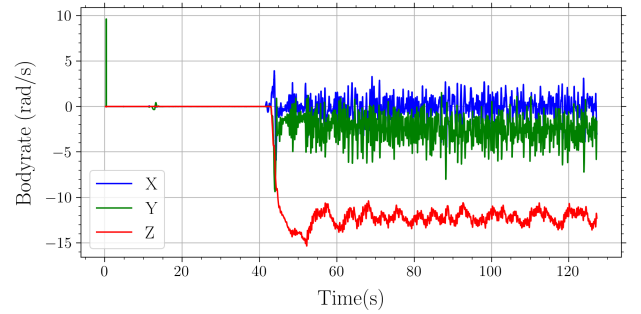


Figure 16. Angular rates after motor failure

9.2.2 Landing Comparison

To compare the performance of the controller, a useful metric is the linear velocity of the drone during the landing routine. The lower the velocity in the z -direction while landing, the better the controller's performance. From Figure 17, we can observe that z velocity before landing becomes < 1 m/s and is approximately 0.5 m/s while landing, which is quite close to 0.4 m/s, velocity in case of the normal controller. Another metric to compare the performance can be to observe z for normal controller and custom controller. It can be seen from Figure 13 and Figure 14 that both the controllers have very similar performance in landing. Note that in the end there are sudden spikes in velocity in z in both normal and failure modes. These occur due to the collision of the drone with the ground in Gazebo.

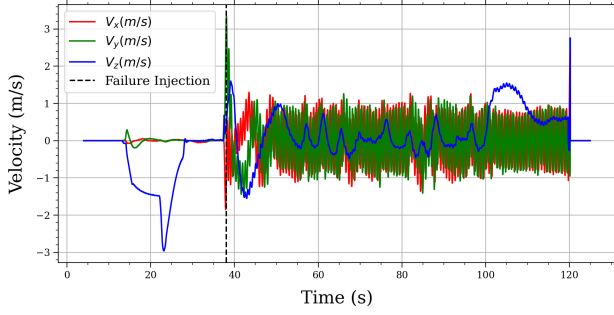


Figure 17. Velocities after motor failure

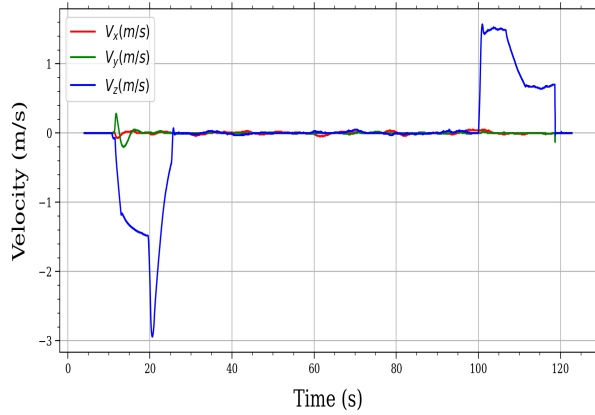


Figure 18. Velocities in normal landing

9.2.3 Navigation Comparisons

To assess the navigation performance, we first need to define a trajectory for the controller to follow. We then plot both the setpoint trajectory and the actual path taken by the drone. As shown in Figure 19, we can see that the drone is able to follow the given set point trajectory well, though the z component is slightly underachieved. A key metric here is the instantaneous error, which is defined as the distance between the current setpoint and the current actual position. We can see in Figure 20 that error < 1.75 m during the entire navigation.

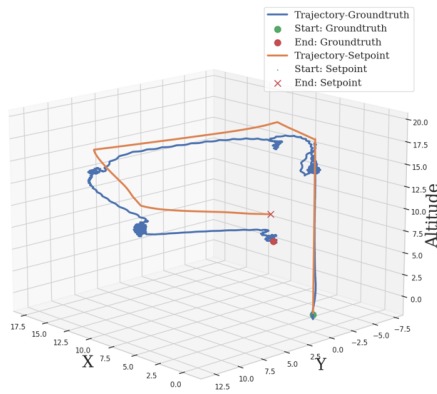


Figure 19. Position in Navigation

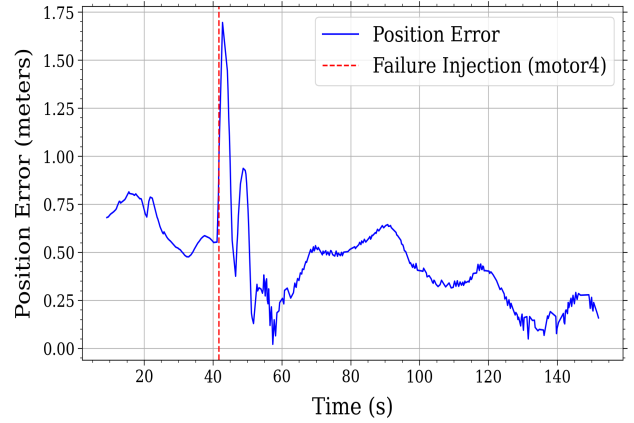


Figure 20. Error in position during navigation

10. Incorporation of Midterm Feedback and Performance Enhancements

The mid-term feedback was instrumental in identifying critical areas for improvement. The modifications outlined in this section directly address the key feedback points and contribute to the refinement of the overall solution.

- The previously utilized Simulink-based jMAVSim framework was discontinued due to its high compilation times and inability to leverage the scope functionality. This limitation arises because the Simulink model is executed on PX4 firmware during the build stage rather than during runtime.
- MATLAB was utilized for testing the failure detection module due to its efficient execution. However, controller implementation and tuning required an environment modeled with realistic wind dynamics and physics to closely replicate real-world conditions, ensuring compatibility with the demands of hardware implementation.
- While implementing the failure detection algorithm in hardware, presence of high sensor noise resulted in false positives detections. To tackle this, we have incorporated a low-pass filter on the measurements received for failure detection to counter the sensor noise.
- Quantitative metrics of the behaviour of the system pre and post failure have been detailed comprehensively in section 9 of this report

Overall, we can see that the controller is able to make the drone hover after the motor failure occurs. The controller is also able to make the drone land with low velocities and is able to perform navigation with minimal inaccuracies in its trajectory.

11. Challenges and Lessons Learned

- **Manual override over failure injection:** While hardware testing (for e.g. parameter tuning), it might happen that the motor in which failure was injected through software, starts behaving abnormally. This might cause the drone to go out of control and can cause damage to the hardware and is dangerous for people in vicinity. We have implemented a manual override functionality that overrides the failure injection so that the pilot can take control of the drone in case of some unexpected behaviour.
- **Setting the desired acceleration in navigation:** The outer controller needs to set the desired acceleration for trajectory following, but inaccuracies in the yaw estimate make it difficult to directly set the acceleration at the center of mass. Instead of using the center of mass, we set the desired acceleration at the drone's center of precision (CoP). Since the inner loop operates at a higher frequency, we assume that precision is already achieved around the CoP. This avoids the need for complex control strategies like bang-bang control, ensuring smoother trajectory tracking without unnecessary circular movements.
- **Calculating parameters for quadrotor hardware:** The moment of inertia around the z-axis (I_Z) was determined using a torsional pendulum setup, while the moments of inertia around the x-axis (I_X) and y-axis (I_Y) were calculated using a simple pendulum setup. The thrust coefficient (K_T) was obtained through the relationship between thrust and weight, where the weight was varied by using different battery configurations.
- **Safety Measures during Hardware Testing:** To ensure the safety of both personnel and equipment, we implemented a drone testing ring made of a protective net (dimensions: 20x20x20 ft) and a cushion bed to absorb impact during falls. Batteries were encased in thick plastic covers to mitigate damage from shock or propeller hits. These precautions allowed for safe testing of the drone's custom firmware and motor failure detection while minimizing risks of hardware or software crashes.
- **Deviations from the referenced control law:** The control law implemented in this work closely follows the one presented in [9], with some key differences. After following the derivation outlined in the paper, the final commanded torques in the x and y directions derived

by us do not match those presented in the original work. These discrepancies may be attributed to differences in conventions used; however, the outputs of our controller align with the expected results.

12. Conclusion

In conclusion, we have successfully achieved the objectives outlined in this report, including motor failure detection, stable hover, navigation, and landing; and have validated our results through simulation. Our implemented motor failure detection algorithm, combined with a fault-tolerant controller, ensures the drone can operate safely even after a motor failure. The controller is capable of adapting to various flight conditions and uncertainties, making it suitable for real-world applications. The integration of our custom controller with the PX4 stack further demonstrates the practical viability of deploying this solution on commercial drones. Additionally, we have identified several areas for improvement and outlined recommendations for future work. Overall, this project has been a valuable learning experience, significantly deepening our understanding of drone control systems and autonomous technologies.

13. Improvements & Applications

Improvements to the Algorithm:

- **Partial Rotor Failure:** The current algorithm can be further enhanced to address partial actuator failures, such as damage to individual propeller blades, rather than complete propeller failure [3].
- **Multi-Rotor Failure Handling:** Extending the current algorithm to manage multirotor failures will significantly improve fault tolerance and expand the system's robustness [15].
- **Sensor Fusion:** Cameras and ultrasonic sensors improve obstacle detection and navigation during single rotor failure by providing real-time feedback for path adjustments. Event cameras help overcome GPS inaccuracies, ensuring better tracking and stability in challenging environments [9].
- **Incorporation of Data-Driven Methods:** With the availability of increased on-board computational power, advanced techniques such as neural networks and reinforcement learning can be integrated to optimize control parameters [16].

- **PID Tuning Using Flight Data:** The log files of simulation and hardware testing can be used to train neural networks for efficient PID tuning.
- **Enhanced State Estimation:** While the current state estimation algorithm performs well, incorporating an Unscented Kalman Filter (UKF) can further improve accuracy non-linearly in nonlinear and noisy environments as compared to EKF [4].

Applications:

- **Military and High-Risk Surveillance:** Designed for critical operations in hostile environments, the drone ensures mission reliability with advanced fail-safes, including an automatic return-to-home function in case of failure. Additionally, it incorporates secure log deletion to safeguard sensitive data and prevent leaks, ensuring both operational success and data protection.
- **Disaster Response in Hazardous Conditions:** In post-disaster scenarios, such as earthquakes or industrial accidents, a fault-tolerant drone can safely operate in environments with debris, extreme temperatures, or high winds. Even with rotor damage, the drone can maintain stability to deliver supplies to map affected areas.
- **Mitigation of External Interferences:** Enhances resilience against bird strikes or similar unexpected events during flight where a partial failure or damage to some component can occur.

References

- [1] K. P. Valavanis and G. J. Vachtsevanos, *Handbook of Unmanned Aerial Vehicles*. Springer Publishing Company, Incorporated, 2014.
- [2] M. H. Amoozgar, A. Chamseddine, and Y. Zhang, "Experimental test of a two-stage kalman filter for actuator fault detection and diagnosis of an unmanned quadrotor helicopter," *Journal of Intelligent & Robotic Systems*, vol. 70, pp. 107–117, 2013.
- [3] Y. Zhang and J. Jiang, "Active fault-tolerant control system against partial actuator failures," *IEE proceedings-Control Theory and applications*, vol. 149, no. 1, pp. 95–104, 2002.
- [4] L. Ma and Y. Zhang, "Dukf-based gtm uav fault detection and diagnosis with nonlinear and lqv models," in *Proceedings of 2010 IEEE/ASME International Conference on Mechatronic and Embedded Systems and Applications*, pp. 375–380, 2010.
- [5] B. Schijndel, S. Sun, and C. Visser, "Fast loss of effectiveness detection on a quadrotor using onboard sensors and a kalman estimation approach," pp. 1–8, 06 2023.
- [6] J. Stephan, L. Schmitt, and W. Fichter, "Linear parameter-varying control for quadrotors in case of complete actuator loss," *Journal of Guidance, Control, and Dynamics*, vol. 41, pp. 1–15, 06 2018.
- [7] A. Marino and F. Neri, "Pid tuning with neural networks," in *Intelligent Information and Database Systems: 11th Asian Conference, ACIIDS 2019, Yogyakarta, Indonesia, April 8–11, 2019, Proceedings, Part I 11*, pp. 476–487, Springer, 2019.
- [8] A. Freddi, A. Lanzon, and S. Longhi, "A feedback linearization approach to fault tolerance in quadrotor vehicles," *IFAC proceedings volumes*, vol. 44, no. 1, pp. 5413–5418, 2011.
- [9] S. Sun, G. Cioffi, C. De Visser, and D. Scaramuzza, "Autonomous quadrotor flight despite rotor failure with onboard vision sensors: Frames vs. events," *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 580–587, 2021.
- [10] "Px4 autopilot user guide." <https://docs.px4.io/main/en/>.
- [11] Setati, Tiro, Botha, Natasha, and Roux, Jeanne Marie, "Experimental approach to calculate the moments of inertia of a hexacopter unmanned aerial vehicle," *MATEC Web Conf.*, vol. 370, p. 05001, 2022.
- [12] "uorb communication." <https://docs.px4.io/main/en/middleware/uorb.html>.
- [13] "Px4 controller diagrams." https://docs.px4.io/main/en/flight_stack/controller_diagrams.html.
- [14] "Nonlinear quadcopter attitude control technical report," 2013.
- [15] M. W. Mueller and R. D'Andrea, "Stability and control of a quadcopter despite the complete loss of one, two, or three propellers," in *2014 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 45–52, 2014.
- [16] P. Sharma, P. Poddar, and P. B. Sujit, "A model-free deep reinforcement learning approach to maneuver a quadrotor despite single rotor failure," *ArXiv*, vol. abs/2109.10488, 2021.

Appendix I

Here, a derivation is provided for commanded torque and thrust expression written in Section 7 using the differential equation for error dynamics given by,

$$\ddot{y}(\tilde{u}) = -\text{diag}(k_p, k_p)y - \text{diag}(k_d, k_d)\dot{y}, \quad (28)$$

where $\tilde{u}$ is the control input, eq. (23) is to be designed to satisfy the above equation.

Assumptions as described in ,

- Since n_{des} is calculated from the outer loop, its rate is slower w.r.t the inner loop dynamics. Thus $\dot{n}_{des} = 0$
- The quadrotor inertia matrix is approximated by $\text{diag}(I_x, I_y, I_z)$
- ω_z and $n_{des,z}$ are constant after rotor failure.
- Propeller gyroscopic moment is negligible compared with rotor thrust and generated torques and hence ignored

$$n_{des}^B = \text{DCM} \cdot n_{des} \quad (29)$$

$$y = \begin{bmatrix} n_x^B - n_{\text{fix},x}^B \\ n_y^B - n_{\text{fix},y}^B \end{bmatrix} \quad (30)$$

$$\dot{n}_{des}^B = \frac{d}{dt}(\text{DCM} \cdot n_{des}) \quad (31)$$

Since $\dot{n}_{des} = 0$,

$$\dot{n}_{des}^B = n_{des}^B \times \omega^B \quad (32)$$

$$\dot{n}_{des}^B = \begin{bmatrix} n_x^B \\ n_y^B \\ n_z^B \end{bmatrix} \times \begin{bmatrix} p \\ q \\ r \end{bmatrix} = \begin{bmatrix} n_y^B r - n_z^B q \\ n_z^B p - n_x^B r \\ n_x^B q - n_y^B p \end{bmatrix} \quad (33)$$

$$\ddot{n}_{des}^B = \frac{d}{dt}(n_{des}^B \times \omega^B) \quad (34)$$

$$\ddot{n}_{des}^B = \dot{n}_{des}^B \times \omega^B + n_{des}^B \times \dot{\omega}^B \quad (35)$$

$$= (n_{des}^B \times \omega^B) \times \omega^B + n_{des}^B \times \dot{\omega}^B \quad (36)$$

Using $\dot{\omega}_z^B = 0$,

$$n_{des}^B \times \dot{\omega}^B = \begin{bmatrix} n_x^B \\ n_y^B \\ n_z^B \end{bmatrix} \times \begin{bmatrix} \dot{p} \\ \dot{q} \\ 0 \end{bmatrix} = \begin{bmatrix} -n_z^B \dot{q} \\ n_z^B \dot{p} \\ n_x^B \dot{q} - n_y^B \dot{p} \end{bmatrix} \quad (37)$$

$$(n_{des}^B \times \omega^B) \times \omega^B = \begin{bmatrix} n_y^B r - n_z^B q \\ n_z^B p - n_x^B r \\ n_x^B q - n_y^B p \end{bmatrix} \times \begin{bmatrix} p \\ q \\ r \end{bmatrix} \quad (38)$$

$$= \begin{bmatrix} n_z^B p r - n_x^B (r)^2 - n_x^B (q)^2 + n_y^B p q \\ n_x^B p q - n_y^B (p)^2 - n_y^B (r)^2 + n_z^B q r \\ n_y^B q r - n_z^B (q)^2 - n_z^B (p)^2 + n_x^B p r \end{bmatrix} \quad (39)$$

y is a 2D vector with no z component thus, combining all the equations and substituting in 28,

$$\begin{bmatrix} n_z^B p r - n_x^B (r)^2 - n_x^B (q)^2 + n_y^B p q \\ n_x^B p q - n_y^B (p)^2 - n_y^B (r)^2 + n_z^B q r \end{bmatrix} + \begin{bmatrix} -n_z^B \dot{q} \\ n_z^B \dot{p} \end{bmatrix} \quad (40)$$

$$= - \begin{bmatrix} k_p \cdot (n_x^B - n_{\text{fix},x}^B) \\ k_p \cdot (n_y^B - n_{\text{fix},y}^B) \end{bmatrix} - \begin{bmatrix} k_d \cdot n_y^B r - n_z^B q \\ k_d \cdot n_z^B p - n_x^B r \end{bmatrix} \quad (41)$$

Equating the x and y components,

$$\dot{q} = [-k_p y_1 + k_d (n_y^B r - n_z^B q) + (n_x^B r - n_z^B r)] \frac{I_y}{n_z^B} \quad (42)$$

$$\dot{p} = [-k_p y_2 + k_d (n_x^B r - n_z^B p) + (n_z^B p - n_x^B r)] \frac{I_x}{n_z^B} \quad (43)$$

Adding the gyroscopic effect terms, we get the desired torque terms :

$$\tau_{x,cmd} = [-k_p y_2 + k_d (n_x^B r - n_z^B p) + (n_y^B r - n_z^B q) r] \frac{I_x}{n_z^B} + I_z q r - I_y q r \quad (44)$$

$$\tau_{y,cmd} = [-k_p y_1 + k_d (n_y^B r - n_z^B q) + (n_x^B r - n_z^B p) r] \frac{I_y}{n_z^B} + I_x p r - I_z p r \quad (45)$$

and commanded torque is

$$T_{cmd} = - \frac{m(a_{z,des} - g)}{\cos \phi \cos \theta} \quad (46)$$

Appendix II

While implementing the Hypothesis test outlined in [5], we came across a slight inconsistency in notation. The test is described as ,

$$P(\hat{k}_i < k_{\text{thres}}) = 1 - \frac{1}{2} \left(1 + \text{erf} \left(\frac{k_{\text{thres}} - \hat{k}_i}{\sqrt{2} \sigma_{\hat{k}_i}} \right) \right) \quad (47)$$

where $P(\hat{k}_i < k_{\text{thres}})$ is the probability of failure of *motor*_i and erf(x) is the Gaussian Error Function Figure 21 varying between [-1,1].

$$\text{erf}(x) = \frac{2}{\sqrt{\pi}} \int_0^x e^{-t^2} dt \quad (48)$$

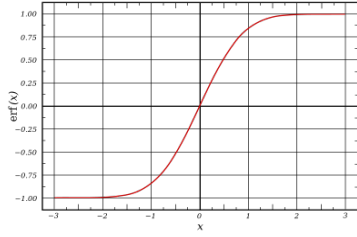


Figure 21. Gaussian Error Function

However according to the standard result, the cumulative distribution function (CDF) is derived from the error function using the following equation,

$$P(k_i < k_{\text{thresh}}) = \frac{1}{2} \left(1 + \text{erf} \left(\frac{k_{\text{thresh}} - k_i}{\sqrt{2}\sigma_{k_i}} \right) \right) \quad (49)$$

This probability varies between 0 to 1 for the given standard definition of erf(x) with a probability of 1 as $\lim_{x \rightarrow \infty}$ and 0 as $\lim_{x \rightarrow -\infty}$. We have implemented this equation in our code for hypothesis test and achieved satisfactory performance as mentioned in section 9

Appendix III

Plots for Motor Failure Detection On Simulation:

Altitude v/s Time Graph At Failure

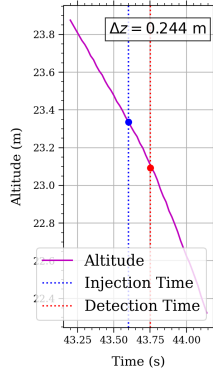


Figure 22. Change in Altitude After Failure in motor 1 from pitch

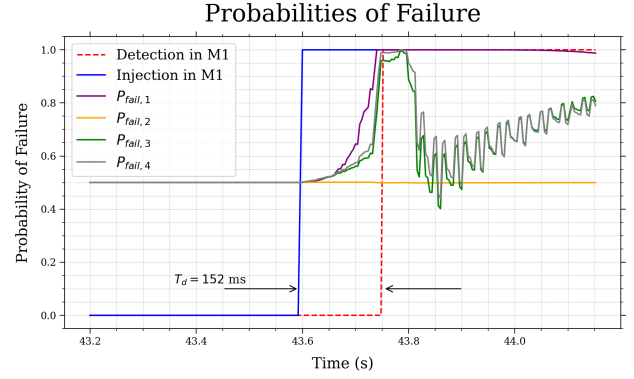


Figure 23. Probabilities of failure in motor 1 from pitch

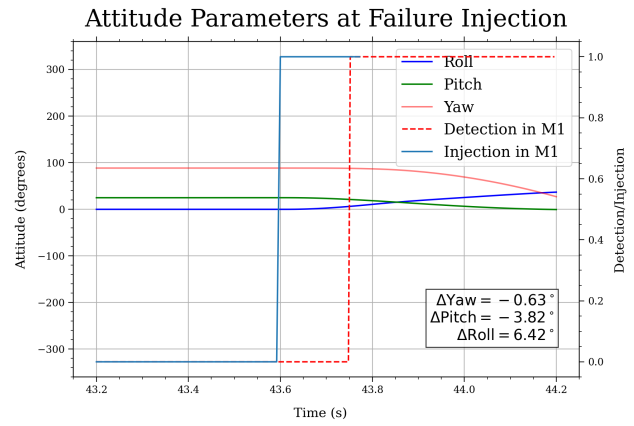


Figure 24. Change in Altitude After Failure in motor 1 from pitch

Altitude v/s Time Graph At Failure

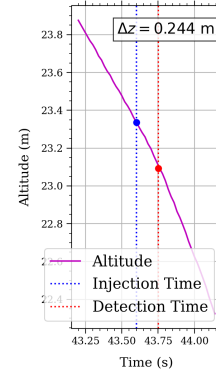


Figure 25. Change in Altitude After Failure in motor 1 from roll

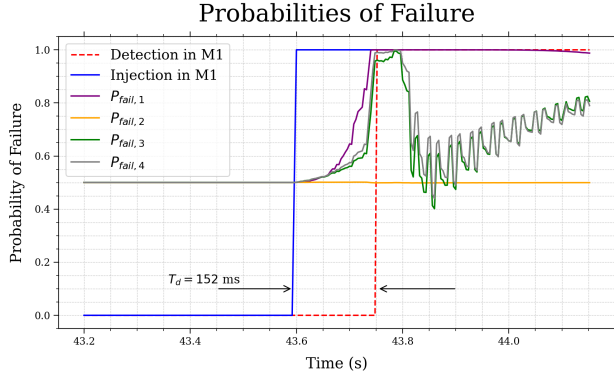


Figure 26. Probabilities of failure in motor 1 from roll

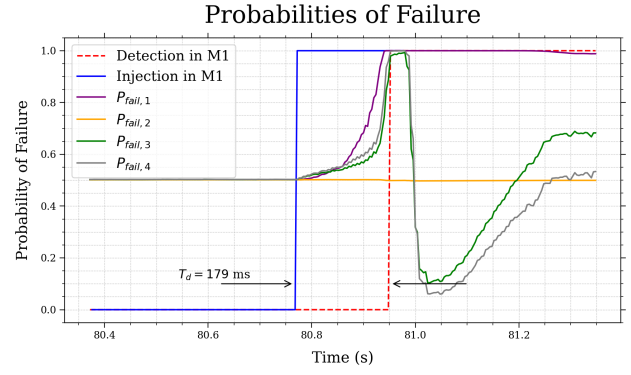


Figure 29. Probabilities of failure in motor 1 from yaw

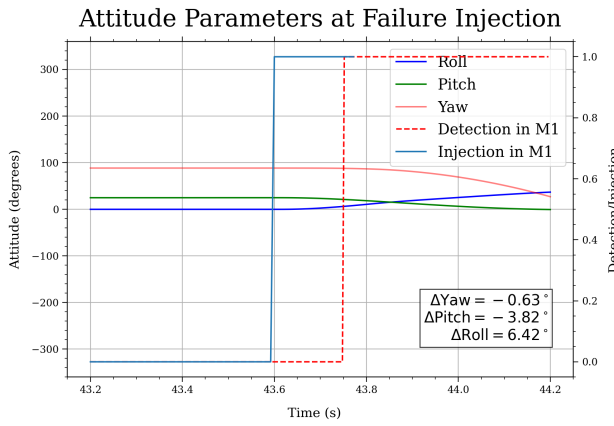


Figure 27. Change in Altitude After Failure in motor 1 from roll

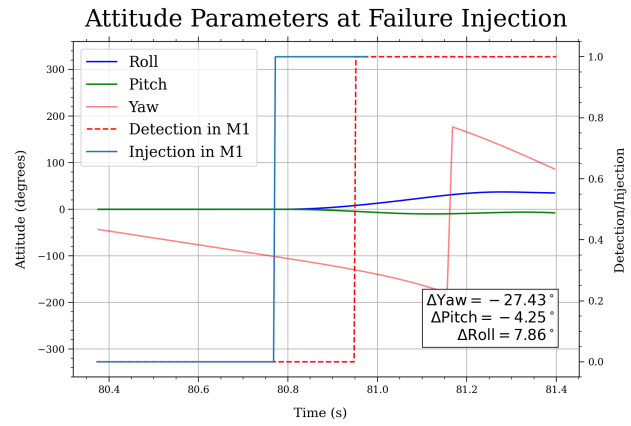


Figure 30. Change in Altitude After Failure in motor 1 from yaw

Hardware:

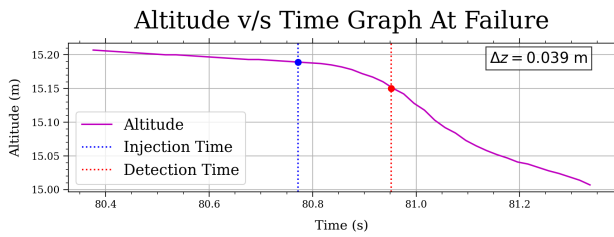


Figure 28. Change in Altitude After Failure in motor 1 from yaw

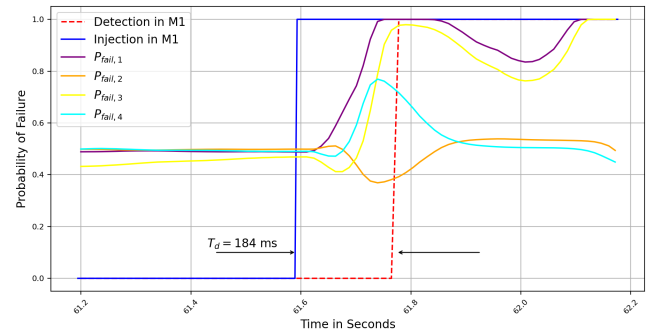


Figure 31. Failure detection when the drone is in the nominal hover state just before failure injection in motor 1 on hardware

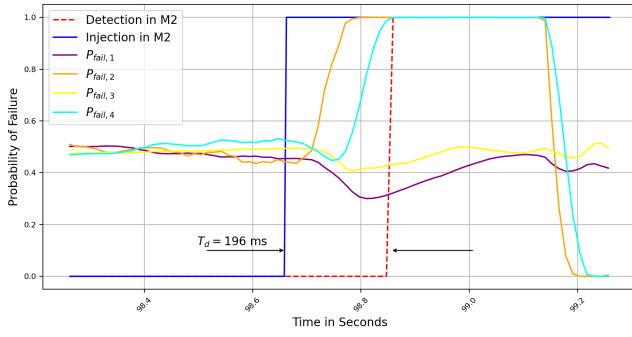


Figure 32. Failure detection when the drone has a non-zero pitch just before failure injection in motor 2 on hardware

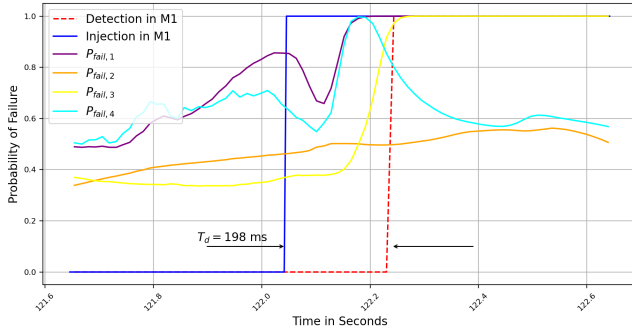


Figure 33. Failure detection when the drone has a non-zero yaw rate just before failure injection in motor 1 on hardware

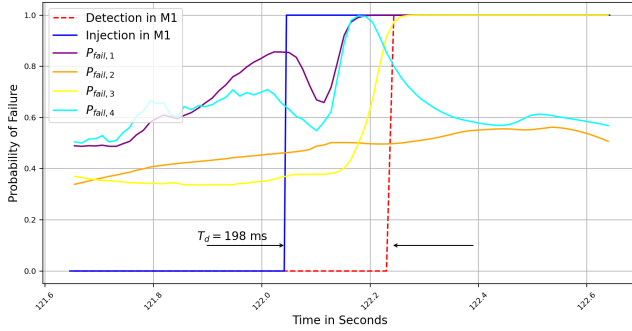


Figure 34. Failure detection when the drone has a non-zero yaw rate just before failure injection in motor 1 on hardware